

Analysis Code Walkthrough

Preoperative muscle morphology and functional recovery after lumbar decompression

Study investigators

2026

Purpose of this document

This document explains the analysis code line by line so that a reader can audit and reproduce every step from the source imaging-and-outcomes dataset to the figures and models reported in the abstract. The code is organised as a small Python package under `src/`. Each module has a single responsibility, and a thin orchestration module (`pipeline.py`) defines the order in which the steps run. No patient data are stored in the repository; the code reads the source workbook from a local path declared in `config.yaml`.

The end-to-end call structure is summarised by the function call graph in Figure 1 and the conceptual design in Figure 2.

Module 1 — `data_loading.py`

This module reads the workbook and converts its awkward two-row header into ordinary column names. The source file labels each anatomical structure on the first header row (for example, “left iliopsoas muscle”) and the per-structure metric on the second row (for example, “Volume (LM) cm3”). The data themselves begin on the third row.

```
def load_config(path: str = "config.yaml") -> dict:
    with open(path) as f:
        return yaml.safe_load(f)
```

`load_config` reads the YAML configuration into a dictionary. Centralising the data path and analysis choices here keeps machine-specific and study-specific settings out of the code, which is what allows the repository to remain free of any absolute paths or data.

```
def load_raw(cfg: dict) -> pd.DataFrame:
    raw = pd.read_excel(cfg["DATA_PATH"], sheet_name=cfg["SHEET"], header=None)
```

We read the worksheet with `header=None` so that pandas does not try to interpret either header row as column names; we will build the names ourselves. `cfg["DATA_PATH"]` and `cfg["SHEET"]` come from the configuration, so the data location is never hard-coded.

```
g = raw.iloc[cfg["HEADER_GROUP_ROW"]].ffill()
m = raw.iloc[cfg["HEADER_METRIC_ROW"]]
```

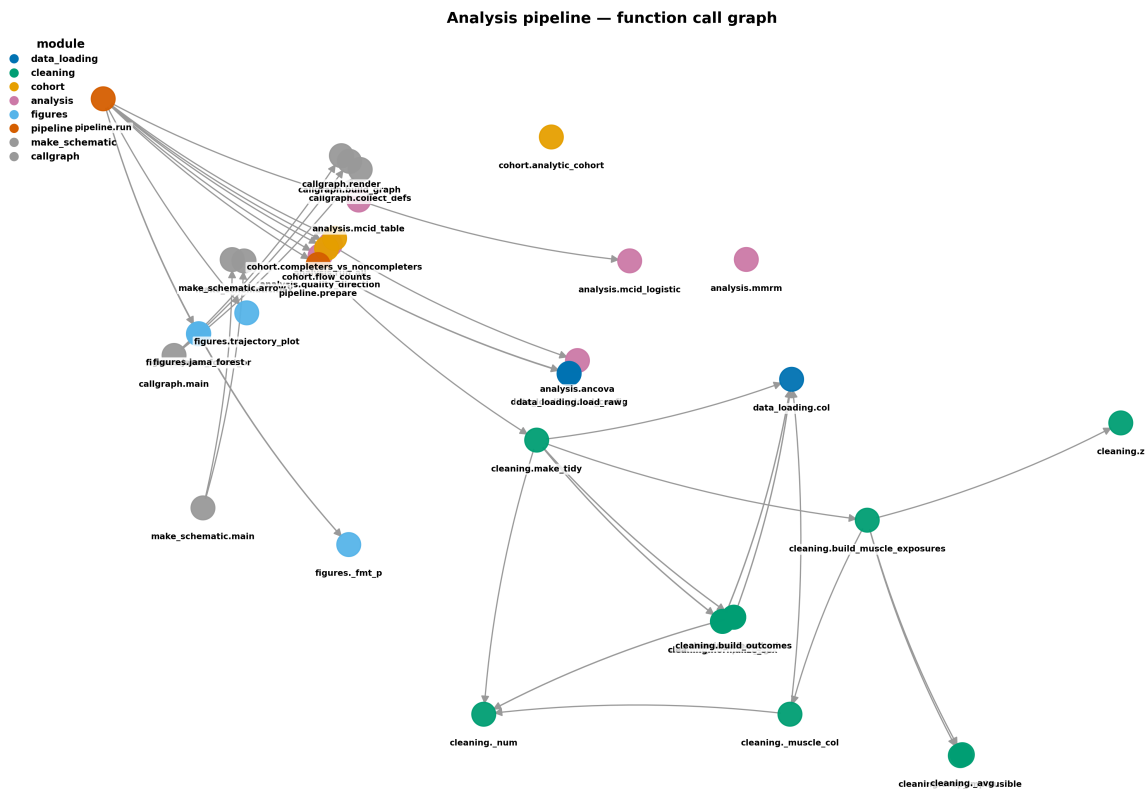


Figure 1: Function call graph of the analysis pipeline. Nodes are functions, coloured by the module that defines them; edges point from caller to callee. `pipeline.prepare` and `pipeline.run` are the orchestration hubs.

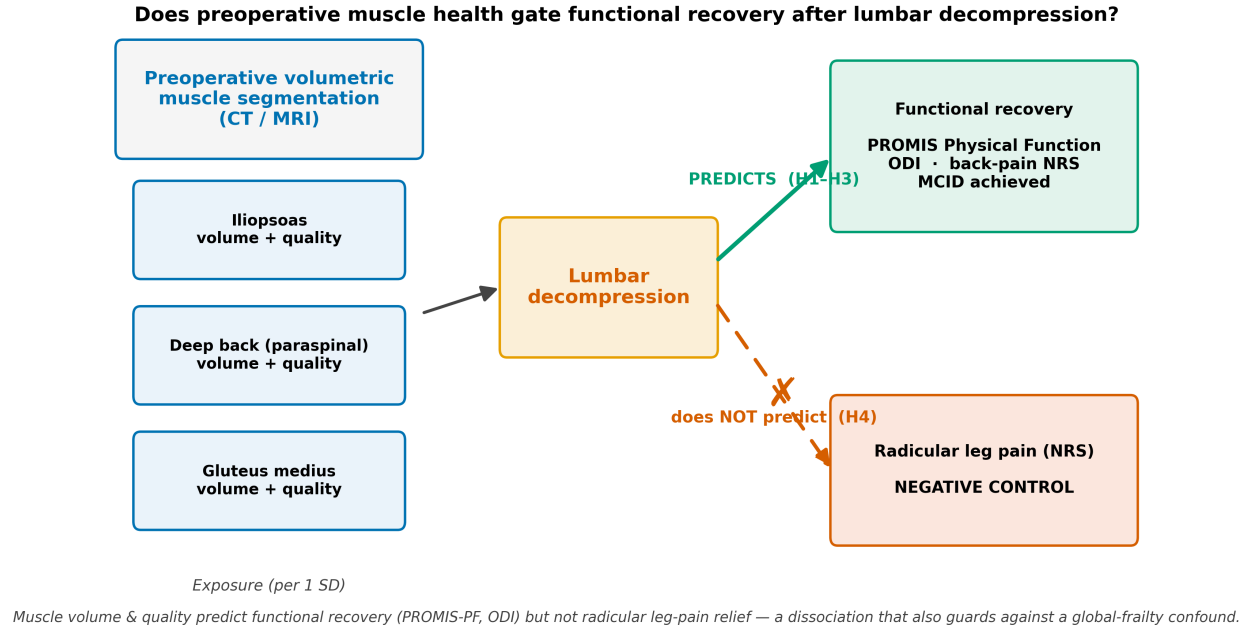


Figure 2: Conceptual study design. Preoperative muscle volume and quality are the exposure; functional outcomes are predicted (solid path), whereas radicular leg pain is a prespecified negative control (dashed path).

The first header row carries the structure label only in its first column and is blank across the remaining columns of that structure's block; `ffill()` (forward fill) propagates each structure label across its block so that every column knows which structure it belongs to. The second row, `m`, holds the metric label for each column.

```
cols = []
for gi, mi in zip(g, m):
    gi = "" if pd.isna(gi) else str(gi).strip()
    mi = "" if pd.isna(mi) else str(mi).strip()
    cols.append((f"{gi} | {mi}").strip(" |"))
```

We pair each structure label with its metric label and join them as "`structure | metric`", cleaning missing values and stray whitespace. The result is a flat, human-readable, unique-enough column name such as "left iliopsoas muscle | Volume (LM) cm3".

```
data = raw.iloc[cfg["DATA_START_ROW"]:].reset_index(drop=True)
data.columns = cols
return data
```

Finally we drop the two header rows, reset the index, attach the constructed names, and return the analysis frame.

```
def col(data: pd.DataFrame, substr: str) -> pd.Series | None:
    hits = [c for c in data.columns if substr.lower() in c.lower()]
    return data[hits[0]] if hits else None
```

`col` is a convenience accessor: because the flattened names are long, we usually want to fetch a

column by a distinctive substring (case-insensitive). It returns the first matching column, or `None` if nothing matches, which keeps downstream code concise.

Module 2 — `cleaning.py`

This module turns raw columns into the variables the models need and addresses the data-quality issues identified during inspection. `compute_age` derives age from the raw datetime components (tolerating corrupted dates that overflow pandas) and restricts to 18–100 years; `normalize_sex` harmonises the mixed male/female/m/f coding; `drop_implausible` sets non-positive values and extreme upper outliers (beyond the third quartile plus three interquartile ranges) to missing, which removes segmentation errors such as the 170,583 cm³ iliopsoas “volume” without distorting the real distribution.

`build_muscle_exposures` is the heart of the module. For each muscle it combines the left and right sides (volume summed; intensity statistics averaged) and constructs two families of exposure. **Size:** total volume and a spine-normalised volume index (muscle volume divided by vertebral-body volume, which removes body-size effects without relying on the corrupted height and weight fields). **Quality:** because absolute MRI signal is not comparable across scanners, we prefer scanner-robust measures of *fatty infiltration* based on intramuscular signal heterogeneity—the coefficient of variation (SD divided by mean) and the normalised interpercentile spread ((p95–p5)/median)—since fatty streaking increases the variance of the muscle signal. These two correlate ~0.9 and are combined into a single `texture` score per muscle. A reference-normalised mean intensity (muscle mean divided by spinal-cord mean) and the raw mean are retained for comparison. Every exposure is z-scored so that estimates are interpretable per standard deviation.

`build_outcomes` extracts baseline through two-year values of each instrument, and `make_tidy` assembles the per-patient frame and flags attainment of the minimal clinically important difference on PROMIS-PF (≥ 4.5) and ODI (≥ 12.8), coding the flag as missing—never zero—when either timepoint is absent.

Module 3 — `cohort.py`

`build_analytic_cohort` restricts the data to patients who have both a muscle segmentation and the requested outcome timepoint, which is the binding constraint on sample size (approximately 120 to 170 patients). `flow_counts` returns the number of patients remaining at each step so that the patient-flow (STROBE) diagram reports exact exclusions.

Module 4 — `analysis.py`

This module contains the statistical models. `ancova` fits the primary continuous analysis—the one-year outcome regressed on a muscle exposure plus the baseline outcome, age, and sex, with HC3 robust standard errors—returning the per-standard-deviation coefficient, confidence interval, and p-value. `mcid_logistic` fits the corresponding logistic regression for MCID attainment, and `mcid_table` assembles the headline contrast of muscle *size* (volume) versus *quality* (texture) across the three muscles for both ODI and PROMIS-PF. `mmrm` fits a linear mixed model over all timepoints to use every patient with at least one follow-up under a missing-at-random assumption. `quality_direction` empirically resolves the sign of the intensity measure by correlating it with

age and volume (older and atrophied muscles show more fat), confirming that higher texture denotes worse muscle quality.

Module 5 — `figures.py`

The figures follow the JAMA convention of a table whose rows carry the estimate, 95% confidence interval, and p-value as text columns, aligned with a forest panel and a null reference line. `jama_forest` renders the continuous (ANCOVA) effects with a reference line at zero; `jama_forest_or` renders the MCID odds ratios on a log scale with a reference line at one and is the headline figure (muscle size versus quality). `trajectory_plot` shows physical-function recovery over time stratified by low versus high iliopsoas fatty infiltration.

Key result

Across 385 imaged patients (mean age 62.8 years, 47% female), muscle **volume** was not associated with achieving MCID for any muscle (all $p > 0.40$). Greater iliopsoas **fatty infiltration** was associated with **lower** odds of achieving MCID on the ODI (OR 0.60 per SD; 95% CI 0.40–0.91; $p = 0.015$) and PROMIS Physical Function (OR 0.70; 0.48–1.02; $p = 0.06$), independent of volume, and specific to the iliopsoas. The interpretation—muscle *quality*, not *quantity*, gates clinically meaningful recovery after decompression—is reported as hypothesis-generating (see `ABSTRACT.md`).

Module 6 — `pipeline.py`

This module wires the others together and defines the order of execution. `prepare` loads the configuration and data and then applies, in sequence, the demographic derivations, volume winsorising, exposure construction, and outcome construction. `run` calls `prepare`, computes the flow counts, builds the analytic cohort, fits all models and sensitivity analyses, and finally writes the figures. Reading `prepare` and `run` top to bottom therefore gives the complete analysis in the exact order it executes, which is the same order shown in the call graph.

Utility modules — `make_schematic.py` and `callgraph.py`

`make_schematic.py` draws the conceptual figure in Figure 2 using only plotting primitives, so it carries no patient data and is fully reproducible. `callgraph.py` parses the package with Python’s `ast` module, records every call from one package-defined function to another, and renders Figure 1; it also writes a Mermaid version (`callgraph.mmd`) that renders directly on the repository page.

How to reproduce

After installing the dependencies and copying `config.example.yaml` to `config.yaml` with the local data path set, the entire analysis runs with:

```
python -m src.pipeline      # full analysis: tables and figures
python src/callgraph.py     # regenerate the call graph
python src/make_schematic.py
```